

Velocity CRM Agent – Single Agentic AI Architecture

1. Concept: One powerful operations agent

Velocity CRM Agent is implemented as a single, powerful agent that owns end-to-end operations intelligence on top of your Zoho-integrated CRM stack (Next.js, FastAPI, PostgreSQL). Instead of multiple specialized agents, one Operations Super-Agent performs bottleneck detection, data quality fixes, lead routing suggestions, sync reliability checks, and insight generation.

2. Responsibilities of the single agent

- Scan pipeline and sync logs to detect operational bottlenecks such as stalled leads, overloaded owners, failing sync jobs, and missing critical fields.
- Propose and optionally execute corrective actions: data clean-up, reassignment of leads, scheduling of follow-ups, and retries of failed syncs.
- Generate narrative insights and summaries for managers, including daily and weekly operations reports.
- Act as a Copilot in the UI, answering "why" questions, explaining decisions, and accepting instructions to tune its behavior.

3. Full-stack architecture

Frontend (Next.js + React + Tailwind)

- Velocity Agent Hub: page where users launch runs of the single Operations Agent and view results.
- Bottleneck Dashboard: visualizations of stalls, overloads, sync issues, and agent actions.
- Copilot Chat: chat interface that talks directly to the single agent for questions, explanations, and manual triggers.

Backend (FastAPI on Cloud Run)

- CRM and Zoho APIs: existing endpoints for reading and updating CRM entities and handling sync jobs.
- Tooling layer: set of well-defined internal tools that the single agent can call (pipeline analysis, data quality checks, routing helpers, sync management, and reporting).
- Agent Orchestrator: one orchestration service that maintains the agent state, implements the perceive → reason → act loop, calls Gemini, and logs runs and actions.

Data layer (PostgreSQL + BigQuery)

- PostgreSQL: operational store for leads, accounts, owners, sync logs, and agent run metadata.
- BigQuery: analytical warehouse storing historical CRM and agent data for deeper trend analysis and dashboards.

Google Cloud AI

- Gemini models via Vertex AI: single reasoning engine for planning, tool selection, bottleneck analysis, and natural language explanations.

4. Tooling layer for the single agent

Pipeline and bottleneck tools

- get_pipeline_snapshot: fetch aggregated stats for leads, opportunities, and owners.
- get_stalled_leads: identify leads with no activity over a configurable number of days.
- get_overloaded_owners: compute owner workloads and detect overload thresholds.
- get_sync_failures: read sync logs and identify recurring failures and rate-limit issues.

Action tools

- update_lead_fields: fix missing or inconsistent data fields on leads and accounts.
- reassign_leads: move selected leads between owners based on capacity and rules.
- schedule_followups: create tasks or reminders for stalled leads.
- retry_sync_jobs: retry failed sync operations using backoff strategies.

Analytics and reporting tools

- query_bigquery: run parameterized SQL against BigQuery to analyze historical performance.
- generate_operations_summary: assemble a structured report object that Gemini turns into a human-readable narrative.

5. Operations Super-Agent loop

Each run of the single agent follows a structured loop:

- 1) Perceive: Call pipeline, sync, and analytics tools to gather the current state and relevant history.
- 2) Reason: Use Gemini to analyze bottlenecks, compare with historical norms, and decide which corrective steps are appropriate.
- 3) Act: Execute action tools (data cleanup, reassignment, follow-up scheduling, sync retries) according to a validated plan.
- 4) Explain: Ask Gemini to summarize findings, actions, and residual risks into a narrative for the UI and dashboards.

5) Learn: Store run and feedback data to refine thresholds, prompts, and rules over time.

6. Guardrails and feedback for a single agent

- Limit the number of entities the agent may modify in one run and require confirmation for large-scale changes.
- Validate Gemini-generated plans against tool schemas before execution, rejecting or correcting unsafe or malformed calls.
- Keep all write actions behind clear roles and permissions, and log every change in an agent actions table.
- Provide a feedback mechanism in the UI for marking runs as helpful or problematic, and allow manual overrides or rollbacks for sensitive changes.

7. Implementation roadmap

Phase 1 – Core tools and data

- Implement pipeline, bottleneck, and sync tools on top of FastAPI, PostgreSQL, and Zoho.
- Create agent_runs and agent_actions tables to track each execution of the single agent.
- Set up BigQuery and basic Looker Studio dashboards using mirrored CRM and agent data.

Phase 2 – Single agent orchestration

- Build the Agent Orchestrator service that defines the perceive → reason → act → explain loop for one Operations Super-Agent.
- Integrate Gemini via Vertex AI, including prompts that describe all tools and constraints.
- Expose an API for launching runs and retrieving findings and summaries.

Phase 3 – Frontend integration

- Create the Velocity Agent Hub page and Copilot chat in the Next.js frontend.
- Display bottleneck findings, actions taken, and narratives for each run.
- Add feedback controls and confirmation flows for high-impact actions.

Phase 4 – Optimization and learning

- Refine thresholds and rules based on feedback and historical data.
- Extend BigQuery analytics and Looker Studio dashboards to monitor agent performance and operational improvements.
- Iterate on prompts and tool definitions to improve reliability and usefulness of the single agent.